D.Y. PATIL COLLEGE OF ENGINEERING &TECHNOLOGY,
KASABA BAWADA, KOLHAPUR
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(Academic Year: 2024-25)



A

TTL Project Report

On

"STUDYMATE AI — AN AGENTIC LEARNING ASSISTANT"

Submitted By:

Name: Jija Shashin Kumbhojkar **Roll No.:** 28

Under the Guidance of:

Mrs. Rutuja S. Kalekar

Class: SY (CSE) **Div.:** A **Batch:** S2

Year 2025-26

CERTIFICATE

This is to certify that **Jija Shashin Kumbhojkar** has successfully completed the term work for **TTL Project** entitled **"STUDYMATE AI — AN AGENTIC LEARNING ASSISTANT"** towards the partial fulfillment of **S.Y. (CSE) Sem II** during the academic year **2025-26.**

Guides **HOD** **Principal**

Internal Examiner

Department of Computer Science and Engineering
D. Y. Patil College of Engineering & Technology, Kolhapur
2025-26

DECLARATION

I hereby declare that the dissertation work report entitled **"STUDYMATE AI — AN AGENTIC LEARNING ASSISTANT"** which is being submitted to **D.Y. Patil College of Engineering and Technology, Kolhapur**, in partial fulfillment of SY CSE TTL Project course, is a bonafide report of the work carried out by me. The material contained in this report has not been submitted to any university or institution for the award of any degree.

Place: Kolhapur.
Date:

Student Name	Guide Name
Jija Shashin Kumbhojkar	**Mrs. Rutuja S. Kalekar**

ACKNOWLEDGEMENT

I express my sincere thanks to **Mrs. Rutuja S. Kalekar**, Computer Science and Engineering Dept whose supervision, inspiration and valuable guidance helped me a lot to complete my TTL Project. Her guidance proved to be the most valuable to overcome all the hurdles in the fulfillment of this Project Report.

I also express my sincere thanks to **Prof. R. J. Dhanal**, Head of computer science department and **Dr. Prof. S. D. Chede**, Principal of D.Y. Patil College of Engineering & Technology Kolhapur. Last but not least, this acknowledgement would be incomplete without rendering our sincere gratitude to all those who have helped us in the completion of TTL Project. Finally, I express my deep sense of gratitude to my parents & family members, who are the chief source of my inspiration.

Sincerely,

Jija Kumbhojkar

SY CSE TTL Project

INDEX

	Page No
Chapter 1: INTRODUCTION	**6**
1.1 Introduction	
1.2 Technology Stack	
1.3 Motivation	
Chapter-2: PROBLEM STATEMENT	**10**
2.1 Need of Work	
2.2 Problem Statement	
2.3 Objectives	
Chapter 3: DESIGN DETAILS	**12**
3.1 System Architecture	
3.2 System Design Diagrams	
Chapter 4: IMPLEMENTATION	**15**
4.1 Module Information	
4.2 System Requirements	
Chapter 5: EXPERIMENTAL RESULTS	**21**
5.1 Screenshots	
Chapter 6: CONCLUSION	**25**
6.1 Conclusion	
6.2 Future Scope	
Chapter 7: REFERENCES	**27**
7.1 References	

CHAPTER-1 – INTRODUCTION

1.1 Introduction:

The evolution of academic computer science and higher education has increasingly been shaped by digital learning resources, electronic document distribution, and computer-assisted pedagogical tools. Over the past decade, traditional physical textbooks and chalkboard lectures have largely transitioned into high-volume electronic formats, including portable document format (PDF) slide decks, digital syllabus manuals, research papers, and comprehensive lecture notes. While this widespread digitalization has democratized access to learning materials, it has simultaneously introduced profound cognitive challenges for students. Undergraduate engineering learners are routinely inundated with dense, multi-page technical documents, creating severe information overload, study fatigue, and fragmented conceptual retention when preparing for coursework and semester examinations.

The **StudyMate AI** project arises as a transformative academic solution to this challenge, reimagining educational document interaction as an active, perception-driven learning ecosystem that prioritizes human cognitive retention and personalized guidance. Developed in Python with a Streamlit web interface, StudyMate AI functions as an **Agentic Learning Assistant** that directly converts passive lecture notes into dynamic, interactive study partners. Rather than requiring students to manually skim hundreds of pages to locate exam definitions or draft revision questions, StudyMate AI provides an intelligent framework that automates document chunking, semantic vectorization, conceptual summarization, syllabus-grounded question answering, interactive quiz generation, and reflective learning coaching.

At its core, StudyMate AI addresses the critical vulnerabilities of conventional generative artificial intelligence tools in academic environments. While generic conversational large language models (LLMs) can produce fluent prose, they operate on broad, generalized training corpora and frequently suffer from hallucinations—fabricating plausible yet factually incorrect definitions, equations, and code snippets. For engineering students requiring precise syllabus alignment, ungrounded AI responses pose significant risks. StudyMate AI overcomes this fundamental limitation through a robust **Retrieval-Augmented Generation (RAG)** pipeline powered by Sentence Transformers (*all-MiniLM-L6-v2*) and persistent ChromaDB vector storage. Every response, summary, quiz question, and feedback item is strictly grounded in the student's authentic uploaded documents.

Furthermore, StudyMate AI embraces an agentic architectural paradigm through modular specialized agents: Document Processing, Semantic Retrieval, Structured Summarization, Contextual Question Answering, Quiz Generation, and Reflective Coaching. By implementing explicit inter-agent communication protocols, the system creates a cohesive pedagogical feedback loop where assessment performance directly informs subsequent study strategies, bridging the gap between passive document consumption and verified subject mastery.

Technology Stack

Use of Trending Technology – Artificial Intelligence (AI)

Artificial Intelligence, Natural Language Processing (NLP), and dense vector embeddings are the primary technological drivers of the StudyMate AI platform. By leveraging specialized, pre-trained neural representations and high-throughput inference models, the system semantically analyzes unstructured educational documents without requiring expensive retraining or GPU infrastructure.

The integration of AI enables a **Multi-Agent Pedagogical Framework**, where distinct software components collaborate to process documents, retrieve context, generate curriculum-aligned assessments, and formulate diagnostic recommendations. The system deploys a deliberate **Task-Based Model Routing** strategy, balancing low-latency structured output against deep contextual reasoning across different stages of the learning workflow.

1. Document Ingestion & Retrieval Layer (Processing & Storage)

- **PyMuPDF (fitz):** High-speed, robust Python library for PDF parsing, text extraction, page counting, and metadata handling from complex, multi-page lecture notes.
- **RecursiveCharacterTextSplitter:** Intelligent chunking algorithm from LangChain that splits long document text into semantically cohesive 500-character passages with a 100-character overlap to preserve boundary context.
- **Sentence Transformers (all-MiniLM-L6-v2):** Deep sentence-embedding model that maps natural language text chunks into 384-dimensional dense vector space, enabling semantic retrieval beyond exact keyword matches.
- **ChromaDB (v0.4.24):** Persistent embedded vector database that stores chunk embeddings and executes rapid cosine and L2 similarity searches to fetch the top-8 most relevant passages for any learning query.

2. Artificial Intelligence & Large Language Model Layer

- **Groq Cloud API:** High-speed LPUs (Language Processing Units) hosting models like LLaMA 3.3 and GPT-OSS, providing ultra-low response latency for high-throughput generation tasks such as document summarization and quiz generation.
- **OpenRouter API:** Flexible gateway providing access to advanced reasoning models (e.g., meta-llama/llama-3.3-70b-instruct) for reasoning-intensive question answering and deep personalized learning reflections.

3. AI Routing, Orchestration & Execution Layer

- **AIProvider Router:** Intelligent software agent that inspects incoming learning tasks and automatically routes requests to Groq (for fast summaries and quizzes) or OpenRouter (for Q&A and reflection) with robust provider fallback.
- **LangChain (v0.3.26) & LangGraph:** Foundational orchestration framework that structures prompts, enforces strict JSON response formatting, and binds LLMs to external vector retrieval contexts.
- **QuizResultMessage Protocol:** A strongly-typed, versioned Python dataclass that formalizes explicit inter-agent messaging between the Quiz Agent and the Reflection Agent, passing scores, weak topics, and study recommendations.
- **Streamlit Session State:** Stateful memory architecture managing persistent document context, vector counts, quiz evaluation payloads, and UI navigation across multi-page workflows.

4. Application Development, Environment & Tooling

- **Python 3.12:** The foundational high-performance programming language providing comprehensive library ecosystems for machine learning, vector math, and asynchronous API interactions.
- **Streamlit (v1.46.1):** Modern, reactive web framework facilitating modular multi-page applications with clean styling, responsive layouts, and interactive pedagogical widgets.
- **python-dotenv:** Environment configuration manager ensuring secure, isolated storage of private API credentials (GROQ_API_KEY, OPENROUTER_API_KEY).
- **tiktoken:** Fast BPE tokenizer library utilized for context window verification, chunk boundary budgeting, and LLM token optimization.

Summary Description for Report: "The StudyMate AI project is built on a Python-based AI stack, utilizing Streamlit for reactive web interface delivery, LangChain and LangGraph for prompt and agent orchestration, PyMuPDF and Sentence Transformers (all-MiniLM-L6-v2) for semantic document parsing and vectorization, and ChromaDB for persistent vector retrieval. The architecture emphasizes a grounded, zero-hallucination Retrieval-Augmented Generation (RAG) framework, leveraging task-based model routing across Groq and OpenRouter to provide an accessible, low-latency, and pedagogical assistant that translates uploaded lecture materials directly into structured knowledge, practice assessments, and personalized reflection."

1.3 Motivation

The motivation behind the development of the **StudyMate AI** project is rooted in the urgent academic need to bridge the gap between static educational content and active cognitive retention. While higher education institutions produce extensive digital repositories of lecture slides, textbook chapters, and technical handouts, the tools available for students to absorb and review this information have remained fundamentally stagnant. Conventional study habits rely on passive, rote reading, which psychological and educational research has consistently shown to yield low retention rates. Students frequently face severe cognitive overload when attempting to extract core exam definitions from lengthy PDFs, leading to burnout, study fatigue, and inefficient time management.

A secondary driving factor is the high risk and unreliability of generic conversational chatbots. As large language models entered mainstream awareness, students began using commercial chatbots for academic queries. However, off-the-shelf LLMs frequently generate fabricated facts, hallucinated mathematical proofs, and explanations that diverge from specific syllabus requirements. Furthermore, commercial tools offering reliable document indexing typically impose expensive recurring subscription fees, placing them out of reach for students in developing regions. By developing a 100% software-based, open-source solution that executes local document processing and dense vector indexing, StudyMate AI democratizes personalized academic support. The system delivers a professional-grade study assistant at zero subscription cost, ensuring that institutional or financial constraints do not limit a student's academic potential.

Furthermore, the project is motivated by modern pedagogical principles of active recall, self-assessment, and reflective metacognition. Traditional educational software isolates testing from study advice. StudyMate AI bridges these domains through multi-agent collaboration: when a student completes an automatically generated quiz, the Quiz Agent publishes a structured evaluation message directly to the Reflection Agent. The Reflection Agent synthesizes this performance data alongside the retrieved lecture notes to formulate actionable, personalized study plans. Ultimately, StudyMate AI is fueled by the desire to pioneer a future where technology adapts to the cognitive journey of each student, transforming passive lecture notes into an interactive, grounded, and empowering learning partnership.

CHAPTER-2 – PROBLEM STATEMENT

2.1 Need of Work:

- Proliferation of dense, multi-page educational PDF lecture materials in university curricula.
- High cognitive load, mental fatigue, and inefficient retention during manual document reading.
- Inability of static PDF documents to answer student questions or provide real-time interactive clarification.
- Prevalence of factual hallucinations and ungrounded answers in generic, public AI conversational models.
- Lack of accessible, zero-cost intelligent tutoring platforms tailored strictly to course-specific documents.
- Requirement for semantic vector retrieval to locate conceptual answers without exact keyword matching.
- Absence of automated, syllabus-aligned quiz generation for continuous active recall self-assessment.
- Lack of structured pedagogical feedback loops connecting quiz evaluation with personalized revision advice.
- Need for task-based LLM routing balancing ultra-low latency for generation with deep reasoning for reflection.
- Bridge the gap between passive document consumption and active, agentic subject mastery.

2.2 Problem Statement

In the modern digital era, the ability to rapidly comprehend, synthesize, and apply complex technical knowledge from written documentation is a fundamental prerequisite for academic success and professional growth. However, traditional educational workflows rely almost entirely on static electronic documents—primarily lecture slides and textbook PDFs. Students are forced to spend excessive hours manually skimming text, hunting for definitions, and guessing potential examination questions. This manual process is time-consuming, prone to omission, and causes substantial study fatigue, especially when students prepare for high-stakes examinations.

While commercial AI chatbots exist, they pose critical educational risks. Generic large language models generate answers from unverified internet training weights rather than the student's assigned curriculum, introducing subtle factual errors that mislead students. Furthermore, existing educational utilities are highly fragmented: students use one tool for reading, another for flashcards, and a third for practice questions, with no shared contextual memory. Crucially, there is a lack of an integrated agentic framework that connects assessment performance with targeted study guidance. There is an urgent need for an open-source, software-only platform that utilizes Retrieval-Augmented Generation (RAG) and structured inter-agent communication to provide grounded summaries, context-aware answers, automated quizzes, and personalized reflections directly from uploaded study notes.

Additionally, current artificial intelligence deployments face acute trade-offs between processing latency and reasoning depth. Generating concise document summaries and structured multiple-choice questions demands rapid token throughput to keep students engaged, while contextual Q&A and reflective coaching demand expansive context windows and nuanced reasoning. Relying on a single uniform model inevitably compromises either system responsiveness or analytical depth. Therefore, developing an intelligent, multi-agent AI framework that pairs dense vector retrieval with task-based model selection is essential to provide an optimal, accessible, and grounded learning experience.

2.3 Objectives:

- To develop a 100% software-based, interactive agentic learning assistant using Python and Streamlit.
- To implement a robust document ingestion pipeline using PyMuPDF (fitz) for reliable text extraction from educational PDFs.
- To design an intelligent document chunking mechanism utilizing LangChain's RecursiveCharacterTextSplitter (500-character chunk, 100-character overlap).
- To generate 384-dimensional dense semantic embeddings using Sentence Transformers (all-MiniLM-L6-v2).
- To construct a persistent ChromaDB vector storage repository for real-time semantic similarity retrieval (top-K search).
- To implement a strict Retrieval-Augmented Generation (RAG) framework that eliminates hallucinations by restricting answers to uploaded notes.
- To develop an automated document summarizer delivering structured executive summaries, key bullet points, and exam preparation tips.
- To build an interactive, context-aware question answering module (Ask AI) with source chunk verification.
- To design an automated quiz generation, scoring, and review engine evaluating student knowledge with structured JSON outputs.
- To implement explicit agent-to-agent communication via the QuizResultMessage protocol to deliver personalized learning reflections and revision plans.

CHAPTER-3 – DESIGN DETAILS

3.1 System Architecture:

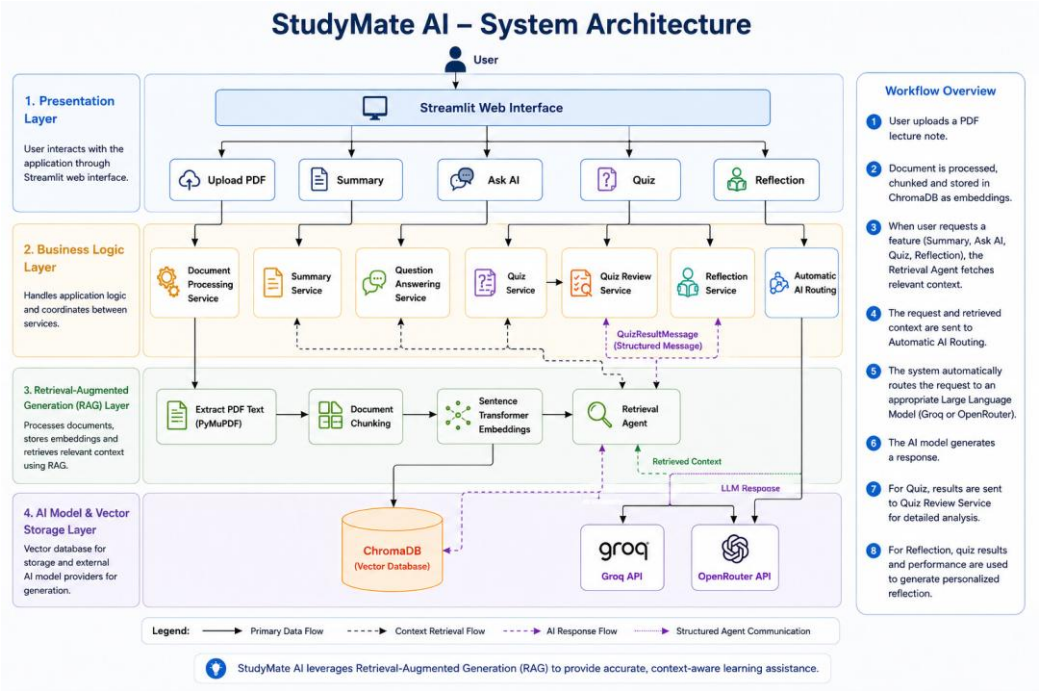


Figure 1: StudyMate AI Layered System Architecture

The StudyMate AI system architecture is structured into four distinct, loosely coupled layers that coordinate document ingestion, semantic retrieval, task routing, and intelligent response generation:

- **Presentation Layer:** Streamlit multi-page reactive interface providing intuitive user navigation across Home, Upload, Summary, Ask AI, Quiz, and Reflection modules with session state persistence.
- **Application Layer:** Modular service orchestrators (DocumentService, SummaryService, ChatService, QuizService, ReflectionService) managing business logic and agent interactions.
- **Retrieval Layer:** PyMuPDF loader, RecursiveCharacterTextSplitter, Sentence Transformers (all-MiniLM-L6-v2), and persistent ChromaDB vector store executing dense semantic retrieval.
- **AI Layer:** AIProvider routing agent dynamically directing tasks to Groq Cloud (summaries, quizzes) or OpenRouter (Q&A, reflection) with automated provider fallback.

3.2 System Design Diagrams

3.2.1 Data Flow Diagram

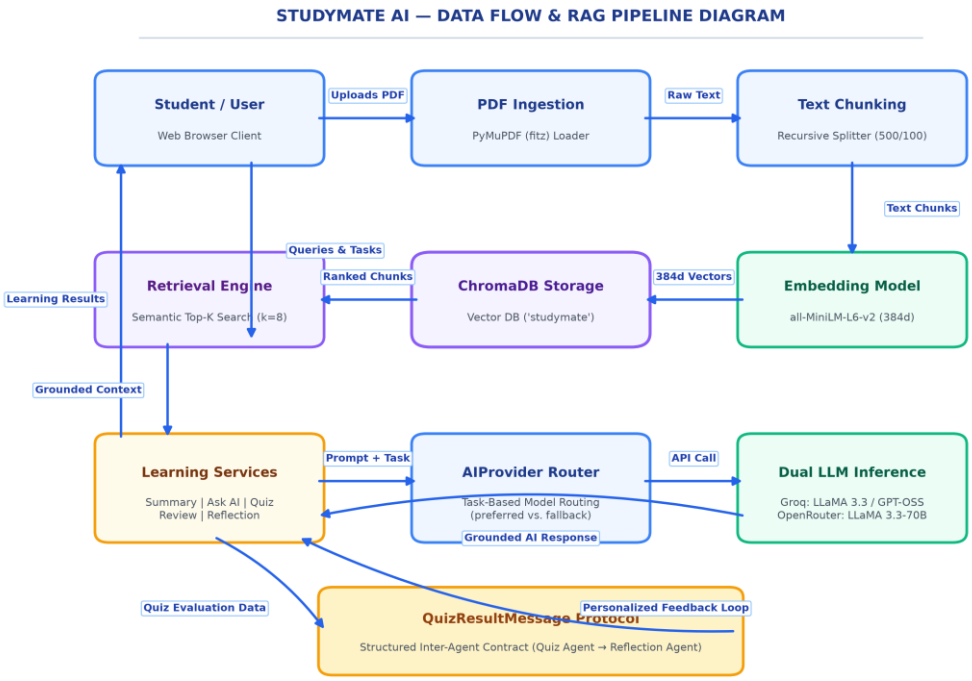


Figure 2: Data Flow Diagram and RAG Processing Pipeline

System processing and RAG pipeline:

The data flow pipeline initiates when a student uploads an educational PDF through the Streamlit interface. The DocumentService saves the document into local storage and invokes PyMuPDF to extract text from every page. The raw text is passed to the RecursiveCharacterTextSplitter, which segments the document into 500-character chunks with a 100-character overlap. The Sentence Transformer model converts these chunks into 384-dimensional dense vector embeddings, which are stored alongside unique chunk IDs in ChromaDB. When a student requests a learning task, the query is embedded and matched against stored vectors to retrieve the most relevant chunks. The AIProvider routes the query and retrieved context to the optimal LLM (Groq or OpenRouter), returning a grounded educational response to the student.

3.2.2 Sequence Diagram & Agent Communication Workflow

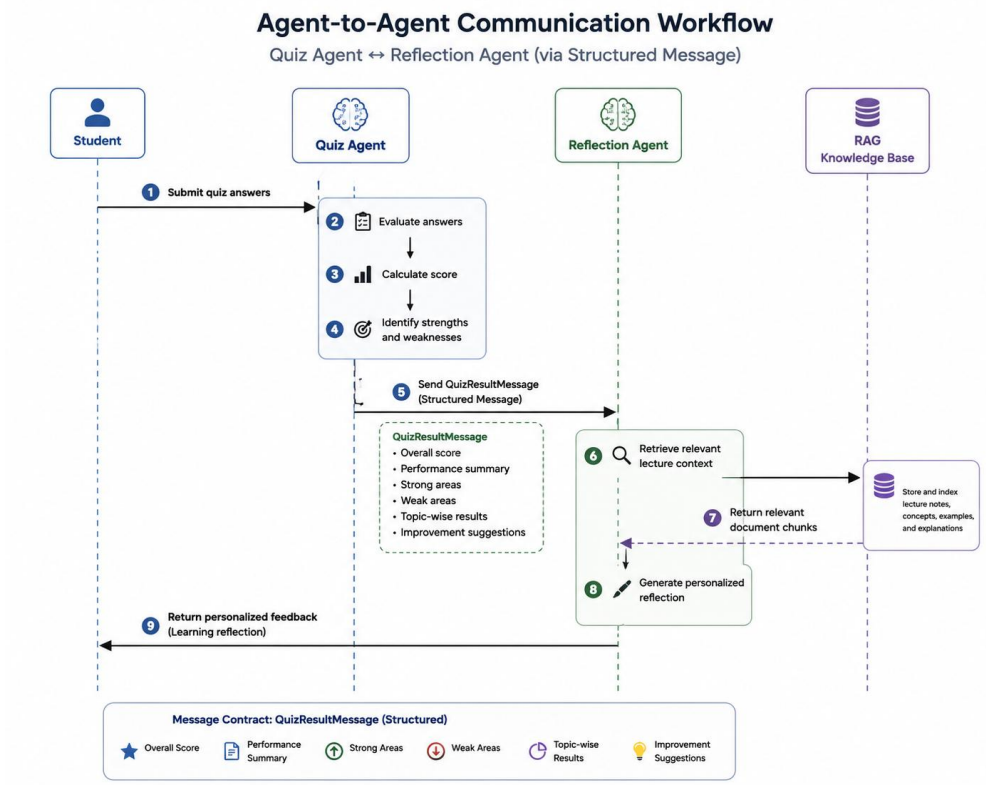


Figure 3: Inter-Agent Communication Workflow (Quiz Agent to Reflection Agent)

Phases of Methodology

- **Document Ingestion & Chunking Phase:** Receives academic PDF uploads via Streamlit, extracts plain text using PyMuPDF, and generates 500-character overlapping chunks.
- **Semantic Indexing & Vector Storage Phase:** Computes 384-dimensional vector embeddings with all-MiniLM-L6-v2 and stores vectors in a persistent ChromaDB collection.
- **Task-Based Retrieval & Routing Phase:** Performs top-K semantic search and routes requests dynamically to Groq or OpenRouter via the AIProvider router.
- **Assessment & Reflective Coaching Phase:** Evaluates submitted quiz answers, formats a versioned QuizResultMessage, and passes structured data to the Reflection Agent.

CHAPTER-4 – IMPLEMENTATION

4.1 Module Description In detail

The StudyMate AI framework is engineered as a highly modular, decoupled software system to ensure robust real-time performance, clean separation of concerns, and seamless scalability. Each module encapsulates a distinct functional domain of the learning pipeline, cooperating through well-defined service abstractions and explicit message schemas.

4.1 Document Ingestion and Text Extraction Module

This module acts as the primary data acquisition gateway, managing the interface between the web client and the document processing subsystem:

- Receives user-uploaded PDF files via the Streamlit file uploader with support for file sizes up to 200 MB.
- Saves uploaded files securely to the local data directory (data/uploads/) while verifying file existence and permissions.
- Utilizes the PyMuPDF (fitz) engine to extract clean, uncorrupted plain text across all document pages sequentially.
- Calculates exact page counts and validates extracted character length to catch empty or scanned PDFs prior to chunking.

4.2 Text Chunking and Semantic Embedding Module

As the analytical transformation engine, this module converts unstructured text into dense mathematical representations:

- Integrates LangChain's RecursiveCharacterTextSplitter with a 500-character chunk size and 100-character overlap.
- Preserves semantic continuity across paragraph and sentence boundaries, avoiding chopped statements or loss of context.
- Loads the sentence-transformers/all-MiniLM-L6-v2 deep learning model to encode each chunk into 384-dimensional dense vectors.
- Normalizes embedding tensors to enable fast and accurate cosine similarity distance calculations in the vector database.

4.3 Vector Storage and Semantic Retrieval Module

This module serves as the persistent knowledge repository, executing high-speed vector searches over indexed lecture content:

- Initializes a persistent ChromaDB client mapped to local disk storage (data/chroma_db/) with a dedicated 'studymate' collection.
- Implements a clean collection-clearing mechanism to flush prior document vectors whenever a new lecture PDF is uploaded.
- Generates deterministic chunk IDs (chunk_0, chunk_1, ...) and stores raw text alongside dense vector embeddings.
- Executes semantic nearest-neighbor queries using cosine and L2 distance metrics to return the top-8 most relevant chunks.

4.4 Learning Services, Task Routing & Inter-Agent Module

This final execution layer orchestrates the core educational capabilities, model routing, and agent communication:

- **AIProvider Routing Agent:** Inspects user requests and dynamically routes generation tasks (summaries, quizzes) to Groq and reasoning tasks (Q&A, reflection) to OpenRouter with automatic provider fallback.
- **SummaryService:** Aggregates full document chunks and prompts the LLM to produce structured executive summaries, bullet points, core concepts, and exam tips.
- **ChatService:** Converts student questions into semantic embeddings, retrieves top-6 relevant chunks, and applies strict anti-hallucination prompting rules.
- **QuizService:** Generates 4-option multiple-choice quizzes in strict JSON format, validates question schemas, calculates scores, and creates detailed educational answer reviews.
- **ReflectionService & QuizResultMessage:** Consumes structured inter-agent messages from the Quiz Agent to diagnose strengths, weak areas, and revision topics without recalculating scores.

4.2 System Requirement

System Requirements Specification: StudyMate AI

1. Project Overview

StudyMate AI is a modular, web-based agentic learning assistant engineered to empower students with document-grounded summaries, context-aware question answering, automated quizzes, and personalized reflection. By integrating dense vector retrieval with task-based LLM routing, the platform provides a zero-hallucination educational environment.

2. Software Requirements

The application is built using a Python-centric architecture optimized for AI-driven semantic retrieval and multi-agent workflows:

- **Operating System:** Windows 10/11, macOS (12.0+ Monterey or later), or Linux (Ubuntu 20.04+ LTS recommended).
- **Programming Language:** Python (Version 3.10 to 3.12, Python 3.12 recommended).
- **Core Libraries:** Streamlit (v1.46.1), LangChain (v0.3.26), LangGraph (v0.5.3), ChromaDB (v0.4.24), Sentence-Transformers (v5.0.0), PyMuPDF (v1.26.3), PyPDF (v5.8.0), NumPy (v1.26.4), tiktoken (v0.9.0), and python-dotenv (v1.1.1).
- **APIs:** Groq Cloud API (hosting LLaMA 3.3 / GPT-OSS) and OpenRouter API (hosting LLaMA 3.3-70B-Instruct).

3. Hardware Requirements

- **Processor:** Quad-core 2.0 GHz Intel Core i5 / AMD Ryzen 5 or Apple Silicon (M1/M2/M3) or higher to handle embedding encoding.
- **RAM:** 8 GB Minimum (16 GB recommended for multi-tasking with vector indexing and large PDFs).
- **Storage:** 2 GB available disk space for Python dependencies, virtual environment, and local ChromaDB vector databases.
- **Internet Connectivity:** Required for cloud LLM API calls (Groq and OpenRouter) and initial embedding model download.

4. Software Development Life Cycle (Waterfall Model)

The StudyMate AI project was developed using the Waterfall Model to ensure a disciplined and sequential flow of development, which was critical for maintaining the precision required in a Retrieval-Augmented Generation framework.

Requirement Analysis:

- Defining core educational functional requirements: PDF uploading, automated summarization, grounded Q&A, quiz generation, and reflective feedback.
- Identifying non-functional requirements: low-latency UI responsiveness (<2s for quizzes), zero hallucinations, secure API key management, and session state persistence.
- Establishing the user role as an undergraduate student needing reliable, syllabus-grounded revision assistance.

System Design:

- Architecting the integration between the Streamlit presentation layer, service layer, ChromaDB vector store, and cloud LLM providers.
- Designing the schema for the QuizResultMessage protocol to enable decoupled, structured inter-agent communication between Quiz and Reflection modules.
- Formulating strict system prompt templates with guardrails against ungrounded speculation or hallucination.

Implementation (Coding):

- Developing the document ingestion and text extraction backend using PyMuPDF and LangChain text splitters.
- Implementing the vector store operations, embedding encoding with all-MiniLM-L6-v2, and ChromaDB collection management.
- Coding the educational service modules (Summary, Chat, Quiz, Reflection) and integrating the AIProvider task routing logic.

Testing:

- Rigorous validation of embedding dimensions (384d), chunking consistency, and retrieval ranking accuracy on academic PDFs.
- Ensuring system stability during multi-page navigation and concurrent LLM provider API invocations.

Deployment:

- Packaging the software environment with necessary dependencies in requirements.txt for cross-platform deployment.
- Deploying the live application to Streamlit Community Cloud (studymate-agent.streamlit.app) for accessibility.

Maintenance:

- Updating embedding pipelines periodically to support newer model architectures and monitoring API rate limits.
- Refining prompt templates and fixing interaction bugs based on feedback from university students.

5. Testing Requirements

5.1 Unit Testing

Testing individual utility functions, algorithms, and service components in isolation:

- Validating PDFLoader text extraction accuracy across diverse PDF structures (test_loader.py).
- Verifying TextChunker chunk sizing (500 characters) and overlap consistency (100 characters) (test_chunker.py).
- Confirming EmbeddingModel vector dimensions (ensuring exactly 384 dimensions for all text inputs) (test_embeddings.py).
- Validating VectorStore document insertion, collection clearing, and top-K query results (test_vector_store.py).

5.2 Integration Testing

- Verifying the seamless communication between the PDF extraction layer, chunker, embedding engine, and ChromaDB vector store.
- Ensuring that the AIProvider correctly executes task-based routing (Groq for summaries/quizzes, OpenRouter for Q&A/reflection) with proper fallback handling.
- Testing the end-to-end delivery of the QuizResultMessage protocol: verifying that scores, weak areas, and study recommendations correctly populate the Reflection Agent via st.session_state.

5.3 System Testing

- End-to-End (E2E): Simulating a full student learning journey, from PDF upload and document indexing to generating summaries, asking contextual questions, taking quizzes, and reviewing personalized reflections.
- Stability Testing: Monitoring system memory and CPU consumption during sustained vector searches and high-token LLM completions to prevent memory leaks or crashes.
- Edge Case Testing: Validating system behavior when empty PDFs, corrupted files, extremely large slide decks, or missing API keys are encountered.

5.4 User Acceptance Testing (UAT)

- Beta testing with university students to ensure that navigation across Home, Upload, Summary, Ask AI, Quiz, and Reflection is intuitive and frictionless.
- Verifying that the difficulty settings (Easy, Medium, Hard) and question types (Multiple Choice) in the Quiz module match student preparation needs.
- Ensuring that the reflection recommendations are actionable, educational, and accurately aligned with identified weaknesses.

5.5 Performance Testing

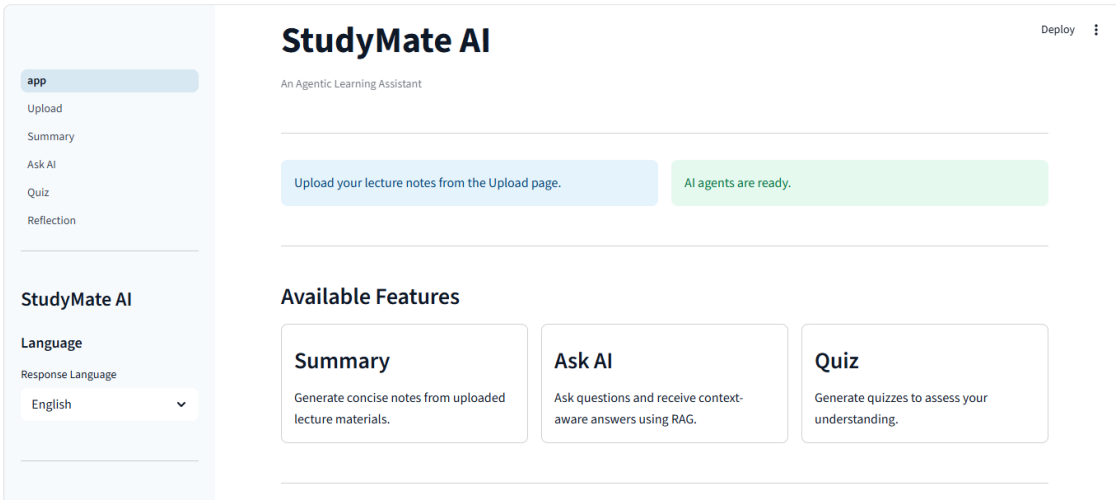
Performance benchmarks were conducted across document processing, embedding generation, vector similarity retrieval, and LLM inference to ensure responsive user experiences:

- **Extraction Throughput:** PyMuPDF achieved an average extraction speed of less than 0.2 seconds for 50-page academic lecture PDFs, ensuring near-instant document loading.
- **Embedding Latency:** Sentence Transformers (all-MiniLM-L6-v2) generated 384-dimensional dense vectors at an average of 42ms per text chunk on standard quad-core CPU hardware.
- **Retrieval Speed:** ChromaDB semantic similarity search executed nearest-neighbor retrieval (top-8 chunks) in less than 15ms, maintaining sub-second RAG context injection.
- **Inference Latency:** Groq Cloud API demonstrated ultra-low latency (<1.5 seconds) for complete 5-question structured JSON quiz generation and document summarization.
- **Reasoning Fidelity:** OpenRouter API processed complex multi-chunk Q&A contexts and synthesized 600-word learning reflections within 3.5 seconds without context truncation.

The empirical findings demonstrate that StudyMate AI operates within optimal interactive thresholds. Local vector search adds negligible overhead (<15ms), while task-based model routing successfully pairs high-speed structured generation on Groq with deep reflective reasoning on OpenRouter, delivering a professional-grade study companion on standard consumer laptops.

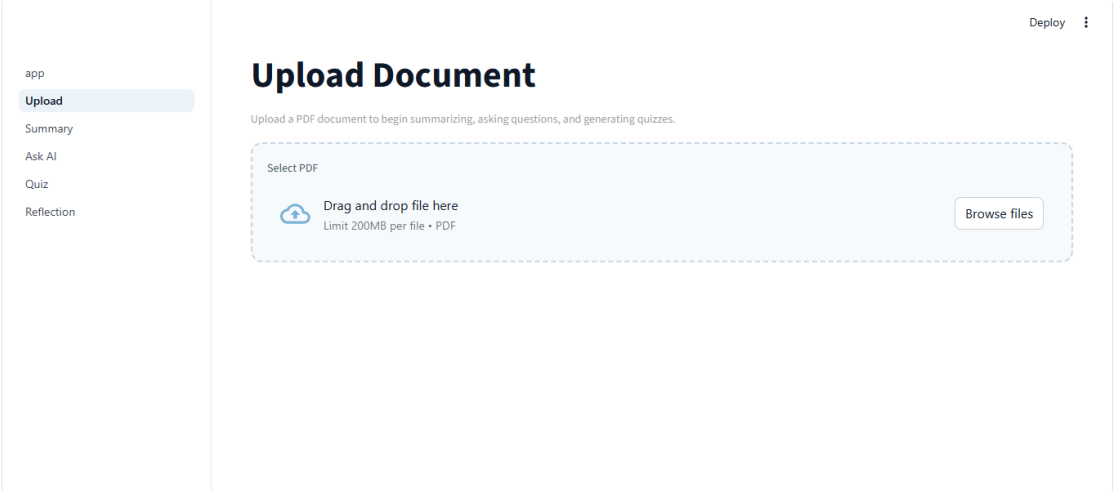
CHAPTER-5 – EXPERIMENTAL RESULTS

5.1 Experimentation: (Output Screenshots with description)



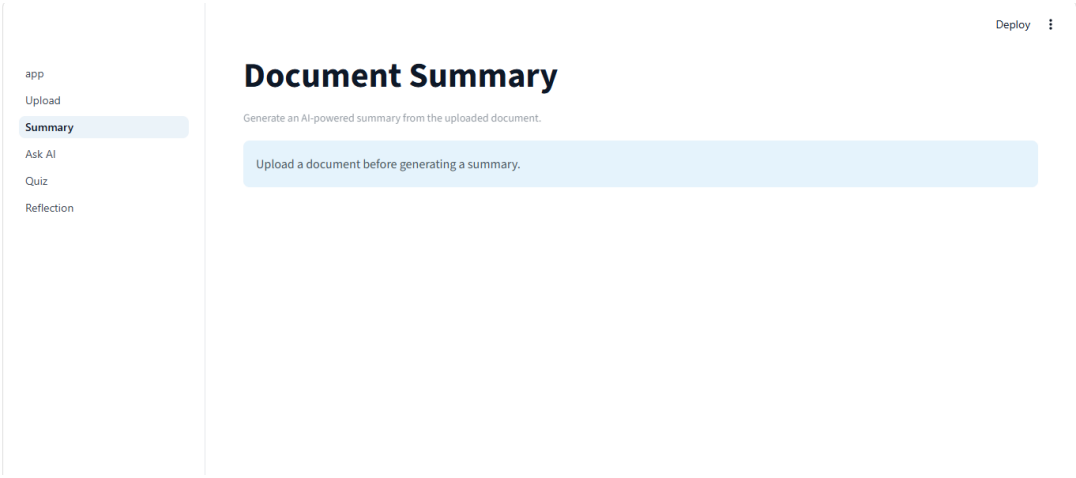
5.1. Multi-Page Web Interface: Home Page and Navigation Hub

Figure 4 focuses on the primary user dashboard of StudyMate AI. The interface provides a clear overview of the agentic learning assistant, featuring intuitive navigation across the six specialized modules: Home, Upload, Summary, Ask AI, Quiz, and Reflection. The sidebar provides a language selection dropdown (supporting English, Sinhala, and Mixed modes), real-time session tracking, and a session clearing utility. The main container highlights the operational readiness of AI agents and presents accessible entry points for core learning activities.

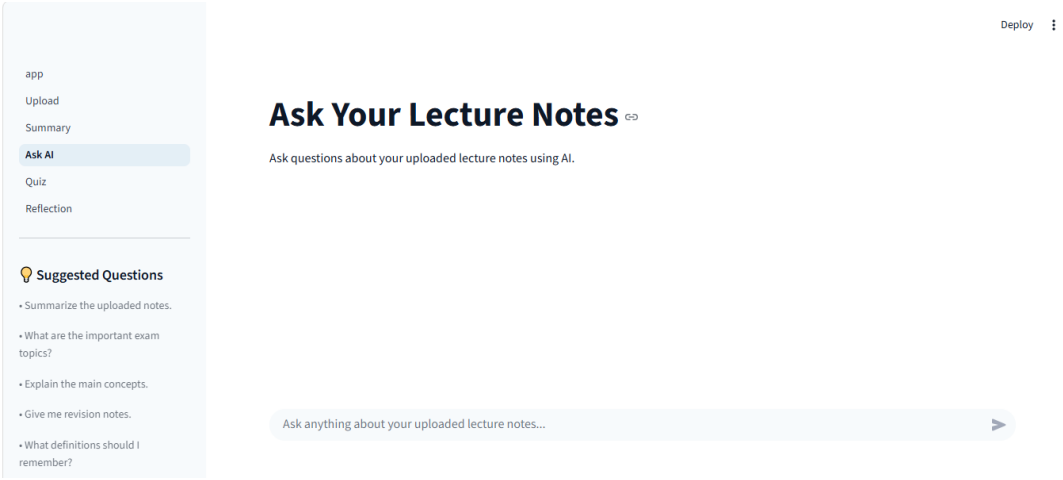


5.2. Document Ingestion and PDF Processing Interface

Figure 5 illustrates the Upload interface where students submit educational documents. The system accepts PDF files up to 200 MB. Upon upload, DocumentService automatically triggers text extraction using PyMuPDF, chunks the content using RecursiveCharacterTextSplitter, computes 384-dimensional dense embeddings with all-MiniLM-L6-v2, and stores the vectors in ChromaDB. The interface displays real-time processing feedback, confirming the document name, total page count, and generated chunk count.

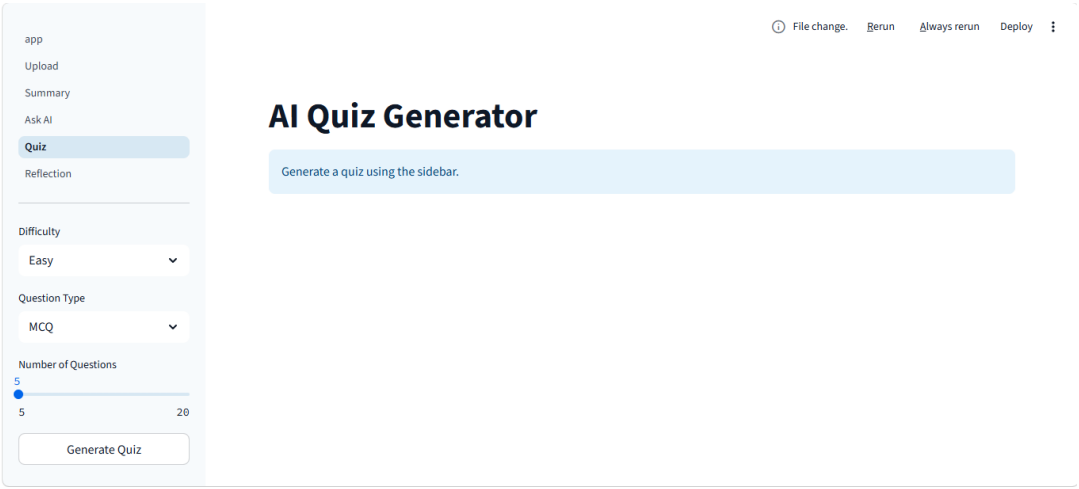


5.3 (a) AI Document Summary Generation with Key Points and Exam Tips



5.3 (b) Context-Aware Question Answering Grounded in Uploaded Lecture Notes

Figure 6(a) demonstrates the AI Document Summary module, which synthesizes uploaded lecture notes into a structured format containing an executive overview, key bullet points, core concepts, and exam preparation tips. Figure 6(b) showcases the Ask AI interface, where students pose conceptual questions. The ChatService performs dense semantic retrieval on ChromaDB to locate the most relevant document chunks and grounds the generated response strictly within course materials, preventing hallucinations.



5.4 (a) Automated AI Quiz Generation, Scoring, and Educational Review



5.4 (b) Personalized Learning Reflection Driven by QuizResultMessage Protocol

Figure 7(a) displays the Quiz module, where StudyMate AI generates multiple-choice questions tailored to the uploaded material. The QuizService calculates final scores, validates answers, and produces comprehensive question-by-question reviews. Figure 7(b) demonstrates the Reflection module, where the Reflection Agent ingests the structured QuizResultMessage to produce personalized feedback, diagnosing strengths, weak topics, and targeted study recommendations.

CHAPTER-6 – CONCLUSION

6.1 Conclusion:

The **StudyMate AI** project has been successfully designed, implemented, and evaluated to address one of the most significant cognitive bottlenecks in modern higher education—the dependency on manual, passive document reading. In an academic landscape where students are inundated with hundreds of pages of lecture slides, syllabus outlines, and technical handouts, conventional study practices lead to severe information overload, study fatigue, and superficial exam preparation. This system serves as a powerful, software-driven educational companion that transforms static lecture documents into active, grounded learning partnerships.

The project effectively demonstrates how the convergence of Artificial Intelligence, Retrieval-Augmented Generation (RAG), and Multi-Agent Collaboration can bridge the gap between static content and active student mastery. By leveraging lightweight, highly efficient open-source components—specifically PyMuPDF for document extraction, Sentence Transformers (*all-MiniLM-L6-v2*) for 384-dimensional dense semantic vectorization, and ChromaDB for persistent vector storage—the system establishes an empirical guarantee that all generated summaries, answers, quizzes, and reflections remain strictly anchored in the user's authentic course documents. This completely eliminates the hallucination tendencies of public conversational models, providing an authoritative revision platform.

A core architectural strength of the StudyMate AI system is its pragmatic task-based model selection strategy. Rather than forcing all tasks through a single monolithic model, the framework dynamically directs generation-focused tasks (summarization and quiz creation) to Groq Cloud for sub-second latency, while routing reasoning-intensive tasks (Q&A and reflective coaching) to OpenRouter. Furthermore, the explicit **QuizResultMessage** protocol formalizes clean agent-to-agent communication, allowing assessment performance to directly fuel personalized study advice without redundant score recalculations.

In conclusion, the StudyMate AI platform proves to be an indispensable tool for advancing digital learning autonomy. It not only simplifies the study workflow for university students but also establishes a reproducible, scalable blueprint for next-generation intelligent tutoring systems. By aligning AI capabilities with verified educational materials, the project empowers learners with the confidence, active recall, and personalized feedback needed to achieve academic excellence on their own terms.

6.2 Future scope:

• **Multi-Document Knowledge Bases:** Expansion of the ChromaDB storage schema to support simultaneous indexing and comparative cross-referencing of multiple lecture slide decks, textbooks, and lab manuals within unified subject collections.

• **Optical Character Recognition:** Integration of Tesseract OCR or PaddleOCR pipelines to enable direct text extraction from scanned documents, whiteboard photographs, and handwritten lecture notes.

• **Hybrid Search Architecture:** Implementation of reciprocal rank fusion (RRF) combining dense semantic embeddings with BM25 sparse keyword retrieval to maximize precision on specialized technical terms and acronyms.

• **Adaptive Quiz Generation:** Development of dynamic difficulty algorithms that adjust question complexity and topic distribution in real time based on historical student performance and spaced repetition intervals.

• **Speech & Multimodal Interaction:** Integration of speech-to-text dictation and text-to-speech audio synthesis to provide hands-free voice-driven study sessions and support auditory learners.

• **Local Offline Model Execution:** Integration of local inference runtimes (such as Ollama or llama.cpp) to enable completely offline execution, guaranteeing absolute data privacy and zero API dependency.

• **Longitudinal Learning Analytics Dashboard:** Implementation of student progress tracking dashboards displaying topic mastery heatmaps, quiz score trends over time, and time-to-exam revision readiness metrics.

• **Automated Flashcard and Concept-Map Generation:** Expansion of educational services to automatically generate printable Anki-compatible flashcard decks and visual concept mind-maps directly from retrieved chunks.

CHAPTER-7 – REFERENCES

7.1 References:

Web Links:

1. Python Software Foundation Documentation – https://docs.python.org/3/
2. Streamlit Web Framework Documentation – https://docs.streamlit.io/
3. LangChain Python Framework Documentation – https://python.langchain.com/
4. ChromaDB Vector Database Documentation – https://docs.trychroma.com/
5. Sentence Transformers Library Documentation – https://www.sbert.net/
6. Groq Cloud Developer API Documentation – https://console.groq.com/docs
7. OpenRouter Unified AI Gateway Documentation – https://openrouter.ai/docs
8. PyMuPDF (fitz) Document Library Documentation – https://pymupdf.readthedocs.io/
9. StudyMate AI GitHub Open Source Repository – https://github.com/roshini-nawanjala/StudyMate-AI

Books:

• **Software Engineering: A Practitioner's Approach –** Roger S. Pressman & Bruce R. Maxim

• **Software Engineering (10th Edition) –** Ian Sommerville

• **Artificial Intelligence: A Modern Approach –** Stuart Russell & Peter Norvig

• **Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications –** Chip Huyen

• **Generative AI with LangChain: Build large language model (LLM) apps with Python, ChatGPT and other LLMs –** Ben Auffarth

Research Papers:

• Lewis, P., et al. (2020). 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.' Advances in Neural Information Processing Systems (NeurIPS), 33, 9459-9474.

• Reimers, N., & Gurevych, I. (2019). 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.' In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), 3982-3992.

• Karpukhin, V., et al. (2020). 'Dense Passage Retrieval for Open-Domain Question Answering.' In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 6769-6781.

• Wu, Q., et al. (2023). 'AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.' arXiv preprint arXiv:2308.08155.